

Formatting Tasks with fm()

by Andri Signorell

Helsana Versicherungen AG, Health Sciences, Zurich

HWZ University of Applied Sciences in Business Administration, Zurich

andri@signorell.net

October 2026

Formatting numbers can be a nightmare in R, and using base resources such tasks can be extremely time-consuming. The reason for this is the multitude of available options (e.g. `format()`, `formatC()`, `sprintf()`, `symnum()`, `prettyNum()`, `strftime()` etc.), which often makes it difficult to find the right one. The functionality often overlaps considerably, lacking consistency. Some functions support certain representations, others do not. Other desirable format options again cannot be found at all.

`fm()` is a general-purpose formatting function for R objects, designed for statistical reporting and publication output. It provides a unified interface for formatting numeric values, dates, text, and tabular objects, with support for reusable format styles.

1	Basic Formatting	2
1.1	Introduction.....	2
1.2	Formatting Numbers	3
1.3	Formatting Dates.....	4
1.4	Special values.....	5
1.5	Alignment	5
1.6	Specific Formats.....	6
1.7	Using Styles.....	8
2	Formatting Higher Dimensional Objects.....	10
2.1	Vectorizing and Objects.....	10
2.2	Combinations of multiple numbers	10
2.3	Matrices and Tables	12
2.4	data.frames.....	12
3	References.....	13

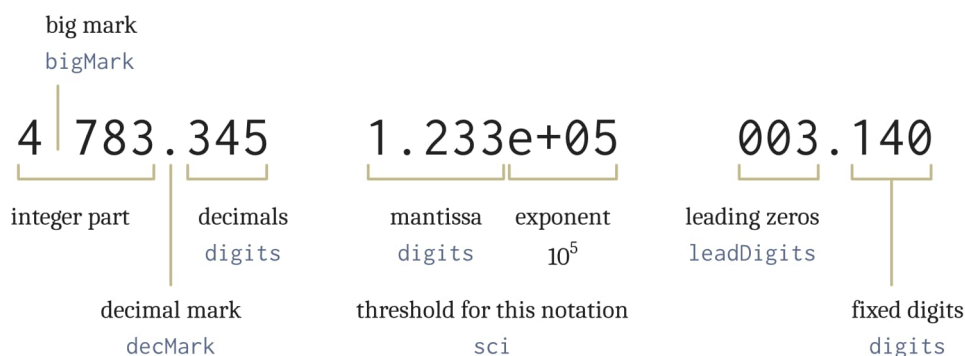
Note: For all the examples in this document, `library(pharos)` must be declared.

1 Basic Formatting

1.1 Introduction

Let us take a quick look at the aspects of representing numbers. Firstly, there is the question of precision. Note that R by default uses “significant” number of digits to display numeric values, which can be set by `options(digits=4)`. Note, that a user may often want to have a FIXED number of digits (consider e.g. p-values, where the decimal places in 1.234e-17 are hardly of any interest).

Next we have to decide when to switch to exponential representation. Base R controls this by `options(scipen=...)`, a penalty that is measured in characters of output width, which makes the switching point hard to predict. `fm()` uses a plain threshold on the absolute value instead: with `sci=7`, the default, a number `x` is displayed in scientific notation if $|x| \geq 10^7$ or $0 < |x| < 10^{-7}$.



We should be aware that formatting means converting a numeric value into a string representation. Calculating with strings can be very troublesome (think of removing big marks). This entails that formatting should only be done after all calculations or modelling processes have been completed. Accordingly, `fm()` returns character values; they have class `noquote` and thus print without quotation marks.

So how can we use `fm()` to get our numbers in shape? Although the function has a large number of arguments, in most cases only a few specific settings are required.

```
fm(x, digits = NULL, leadDigits = NULL, sci = NULL,
   bigMark = NULL, decMark = NULL,
   naForm = NULL, zeroForm = NULL, fmt = NULL, pThreshold = NULL,
   width = NULL, align = NULL, lang = NULL, ...)
```

Only `x` is required. It can be a numeric, logical, character or factor vector, a Date or POSIXt object, or a matrix, table, ftable or data frame (see chapter 2). All format arguments default to `NULL`, which stands for “not specified”: the setting is then taken from a style or from an option, or a built-in default applies. The defaults are given with the arguments below, the order of precedence in 1.7.

`fm()` is the successor of `Format()` in **DescTools**. Readers who know that function will find the interface familiar, but several arguments have been renamed. The old names are no longer accepted:

Format() in DescTools	fm() in pharos
big.mark	bigMark
outdec	decMark
ldigits	leadDigits
na.form, zero.form	naForm, zeroForm

<pre>.. DescTools eps sci = c(lower, upper) lang = "engl"</pre>	<pre>.. pharos pThreshold (default 0.001) sci, one value that is used for both sides lang = "en", which is now the default</pre>
--	--

1.2 Formatting Numbers

The first example addresses the number of digits, the transition to scientific representation and the use of big marks. It uses a space as big mark and 3 fixed digits for all numbers, flips to scientific notation for absolute values $< 10^{-4}$ and $\geq 10^4$ and aligns the result on the decimal point.

```
x <- pi * 10^(-6:6)
writeLines(fm(x, digits=3, sci=4, bigMark=" ", align="."))
##      3.142e-06
##      3.142e-05
##      0.000
##      0.003
##      0.031
##      0.314
##      3.142
##     31.416
##    314.159
##   3 141.593
##      3.142e+04
##      3.142e+05
##      3.142e+06
```

The arguments involved are:

x	a numeric, logical, character or factor vector, a Date or POSIXt object, or a matrix, table, ftable or data frame. Character, factor and logical values are returned as text without any numeric formatting, factors by their labels. <code>naForm</code> , <code>width</code> and <code>align</code> are applied to them as well.	
digits	<i>integer</i> , the desired (fixed) number of digits after the decimal point. You will always get this number of digits, even if the last digit is 0. Ties are rounded away from zero, where <code>round()</code> would round to the even digit. Negative numbers of digits round to the specific power of ten (<code>digits=-2</code> would round to the nearest hundred). If <code>digits</code> is not given, all values get as many decimals as the value with the most decimals requires (at 7 significant digits).	<pre>fm(3.141593, digits=3) ## 3.142 fm(0.031415, digits=3) ## 0.031 fm(3.10012, digits=3) ## 3.100 fm(3142.2, digits=-2) ## 3100 fm(c(0.5, 2.5, -1.5), digits=0) ## 1 3 -2 fm(c(3.1, 2.25, 10)) ## 3.10 2.25 10.00</pre>
leadDigits	<i>nonnegative integer</i> , the minimum number of digits before the decimal point. <code>leadDigits=3</code> makes sure that at least 3 digits are printed on the left side. Setting <code>leadDigits=0</code> removes the leading zero for values in the interval $(-1, 1)$ and yields a result like <code>.452</code> for <code>0.452</code> . The default <code>NULL</code> leaves the numbers as they are (meaning at least one digit).	<pre>fm(3.1415, leadDigits=3, digits=2) ## 003.14 fm(0.3141, leadDigits=0, digits=2) ## .31</pre>
sci	<i>number</i> , the power of 10 at which the display switches to exponential notation. The threshold applies to the absolute value and to both sides: with <code>sci=4</code> , a number <code>x</code> is displayed scientifically if $ x \geq 10^4$ or $0 < x < 10^{-4}$. <code>digits</code> then defines the number of decimals of the mantissa. The default is <code>getOption("scipen")</code> , where R's preset <code>scipen=0</code> is replaced by 7.	<pre>fm(c(1234567, 12345678), digits=1) ## 1234567.0 1.2e+07 fm(c(1234.5, 123456.7), digits=1, sci=4) ## 1234.5 1.2e+05 fm(0.0000271, digits=2, sci=4) ## 2.71e-05</pre>

bigMark	<i>character</i> , used as mark between every 3 digits before the decimal point. The default is <code>getOption("bigMark")</code> or, if this option is not set, "" (none).	<pre>fm(3141, bigMark=" ") ## 3 141 fm(3141.12, bigMark=",", digits=0) ## 3,141 fm(1234567.891, bigMark=" ", digits=2) ## 1'234'567.89</pre>
decMark	<i>a single character</i> , specifying the decimal mark to be used. If not provided, the default is given by <code>getOption("OutDec")</code> .	<pre>fm(3141.593, bigMark=" ", decMark=",") ## 3'141,593</pre>

1.3 Formatting Dates

The argument `fmt` can be used to format `Date` and `POSIXct` objects using a custom, ISO-8601-inspired token syntax similar to .NET or Moment.js.

These format codes (e.g. `d`, `M` and `y` for day, month or year) are more intuitive than the C format codes. Repeating the specific code defines the degree of abbreviation. So the format `yyyy-MM-dd` would yield a date as 2020-10-12. Weekdays and month names can be displayed in English or in the current locale.

```
x <- as.Date(c("2026-01-02", "2018-10-17"))
fm(x, fmt="...")
```

Code	Description	Example with <code>fmt = ...</code>
d	day of the month without leading zero (1 - 31)	<pre>d.MM.yyyy ## 2.01.2026 17.10.2018</pre>
dd	day of the month with leading zero (01 - 31)	<pre>dd.MM.yyyy ## 02.01.2026 17.10.2018</pre>
ddd	abbreviated name for the day of the week (e.g. Mon)	<pre>ddd, dd.MM.yy ## Fri, 02.01.26 Wed, 17.10.18</pre>
dddd	full name for the day of the week (e.g. Monday)	<pre>dddd ## Friday Wednesday</pre>
do	day of the month with English ordinal suffix (1st, 2nd, 3rd, 4th). For most other languages ordinal dates are written with punctuation, e.g. <code>d. MMM yyyy</code> .	<pre>MMM do, yyyy ## Jan 2nd, 2026 Oct 17th, 2018</pre>
M	month without leading zero (1 - 12)	<pre>d.M.yyyy ## 2.1.2026 17.10.2018</pre>
MM	month with leading zero (01 - 12)	<pre>yyyy-MM-dd ## 2026-01-02 2018-10-17</pre>
MMM	abbreviated month name (e.g. Jan)	<pre>d. MMM yyyy ## 2. Jan 2026 17. Oct 2018</pre>
MMMM	full month name (e.g. January)	<pre>d. MMMM yyyy ## 2. January 2026 17. October 2018</pre>
y	year without century, without leading zero (0 - 99)	<pre>d/M/y ## 2/1/26 17/10/18</pre>
yy	year without century, with leading zero (00 - 99)	<pre>dd.MM.yy ## 02.01.26 17.10.18</pre>
yyyy	year with century	<pre>y <- as.POSIXct("2026-01-02 21:14:12", tz="Europe/Zurich") fm(y, fmt="...") H:mm ## 21:14 HH:mm:ss ## 21:14:12</pre>
H/HH	hour in 24h format, one digit / two digits	

h/hh	hour in 12h format, one digit / two digits. In this case t or tt must be set as well to ensure uniqueness, otherwise an error is raised.	h:mm tt ## 9:14 PM
t/tt	AM/PM designator (one/two characters)	hh:mm:ss t ## 09:14:12 P
m/mm	minutes, one digit / two digits	dd.MM.yyyy HH:mm ## 02.01.2026 21:14
s/ss	seconds, one digit / two digits	
lang	<i>character</i> , the language for the names of months and weekdays. Can be either "en" for English or "local" for the current locale. If left to NULL, the option DescToolsX.lang is used and, if this is not set, "en".	fm(x[1], fmt="dddd, d. MMMM", lang="local") ## in a German locale: ## Freitag, 2. Januar

Mind the case: M stands for the month, m for the minutes, so that dd.mm.yyyy returns 02.00.2026 for the first date above. Punctuation and blanks in the format string are copied as they are, but there is no way to include literal text: a letter that is not a format code raises an error. A POSIXct value is displayed in its own time zone, just as format() does.

Properties of a date such as the year, the month or the weekday are, in the broadest sense, just another representation of it. As they often form the basis for further calculations, **DescToolsX** provides them as functions rather than as formats: day(), month(), year(), quarter(), week(), isoYear(), weekday(), dayOfYear(), yearMonth(), yearWeek() and zodiac() for dates, hour(), minute(), second() and timezone() for times.

1.4 Special values

Missing values sometimes require special representation. This is made possible by the argument naForm. All missing values are given the specified form as soon as the argument is assigned. The same applies to 0 values with the argument zeroForm.

naForm	<i>character</i> , string specifying how NAs should be specially formatted. If set to NULL (default) no special action will be taken and missing values remain NA. NaN is treated like NA.	fm(c(1, NA, 8), naForm=".", digits=0) ## 1 . 8 fm(c(1, NaN, 8), digits=0) ## 1 <NA> 8
zeroForm	<i>character</i> , string specifying how zeros should be specially formatted. Useful for pretty printing 'sparse' objects. If set to NULL (default) no special action will be taken. Only values that are zero are replaced, not those that merely round to zero.	fm(c(1, 0, 8), zeroForm="-", digits=0) ## 1 - 8 fm(c(0.0001, 0), zeroForm="-", digits=2) ## 0.00 -

Infinite values are shown as Inf and -Inf, whatever the number format is. A negative value that rounds to zero is shown without a sign, there is no "negative zero":

```
fm(c(-0.0001, Inf, -Inf), digits=2)
## 0.00 Inf -Inf
```

1.5 Alignment

Alignment can be challenging. A simple and robust approach in console-based environments that use fixed-width (monospace) fonts is to rely on spaces for alignment, ensuring predictable and reproducible output across platforms and devices.

The following arguments provide fine-grained control over how character strings are aligned and displayed within a fixed width, making it straightforward to produce neatly formatted console output.

align the character on whose position the strings will be aligned. Left alignment can be requested by setting `align = "\\l"`, right alignment by `"\\r"` and center alignment by `"\\c"`. Mind the backslashes, as if they are omitted, strings would be aligned to the **character** l, r or c respectively. The default is NULL which would just leave the strings as they are.

This argument is sent directly to `strAlign()` as argument `sep`.

width *nonnegative integer*, the minimum width of the strings. Shorter strings are padded with blanks: on the left by default, on the right for `align = "\\l"` and on both sides for `"\\c"`. Longer strings are never truncated.

The examples are wrapped in `cbind()` in order to show the elements one below the other and to make the padding visible:

```
cbind(fm(cumsum(10^(0:6)), align="\\c", digits=0))
```

```
##      [,1]
## [1,] "   1  "
## [2,] "  11  "
## [3,] " 111  "
## [4,] "1111  "
## [5,] "11111 "
## [6,] "111111"
## [7,] "1111111"
```

```
cbind(fm(c(0.5, 12, 1234.5), digits=1, width=10))
```

```
##      [,1]
## [1,] "    0.5"
## [2,] "   12.0"
## [3,] "  1234.5"
```

```
cbind(fm(c("yes", "no", "unknown"), width=10, align="\\l"))
```

```
##      [,1]
## [1,] "yes   "
## [2,] "no    "
## [3,] "unknown"
```

1.6 Specific Formats

Sometimes more variability is needed to display numeric values. For such cases of more specific formatting of numerical values, there is the `fmt` argument. It is very flexible and is used to generate a variety of different formats, such as percentages, engineering representation or p-values. If `x` is a date it serves for the specific date/time format codes (see 1.3).

fmt one of the following:

a *character* string, interpreted as format code (see below) or, for dates, as date format,

the name of a style or an object of class `Style`, consisting of a list of arguments accepted by `fm()` (see 1.7),

a *function* that takes the values and returns one string for each of them.

pThreshold *number*, the reporting threshold for p-values. Values less than `pThreshold` are formatted as `"< [pThreshold]"`. Default is 0.001.

For the most frequently used formats there are the following special codes available:

%	percent multiplies the given number by 100 and appends the %-sign (without a separator). Digits will be set to 1, if not provided otherwise.	<code>fm(0.1234, fmt="%")</code> ## 12.3% <code>fm(0.1234, fmt="%", digits=0)</code> ## 12%
e	scientific forces scientific representation of <code>x</code> , e.g. 3.141e-05. digits gives the number of decimals of the mantissa.	<code>fm(31415.9, fmt="e", digits=3)</code> ## 3.142e+04

p	p-value formats the values as p-values. <code>pThreshold</code> defines the threshold to switch to a < 0.001 representation (more detailed comments below).	<pre>fm(0.0312, fmt="p") ## 0.031 fm(0.00034, fmt="p") ## < 0.001 fm(1.2, fmt="p") ## <NA></pre>
*	significance will produce a significance representation of a p-value consisting of * and ., while the breaks are set according to the used defaults e.g. in <code>lm()</code> as [0, 0.001] = *** (0.001, 0.01] = ** (0.01, 0.05] = * (0.05, 0.1] = . (0.1, 1] = blank The result is padded with blanks to three characters.	<pre>fm(0.082, fmt="*") ## . fm(0.003, fmt="*") ## **</pre>
p*	p-value AND stars will produce p-value and significance stars	<pre>fm(0.003, fmt="p*") ## 0.003 **</pre>
eng	engineering forces scientific representation, restricting to powers that are a multiple of 3.	<pre>fm(1234567, fmt="eng", digits=2) ## 1.23e+06 fm(0.00012, fmt="eng", digits=0) ## 120e-06</pre>
engabb	engineering abbreviation same as eng, but replaces the exponential representation by the SI prefix, e.g. M for mega (1e6). See the data set <code>Prefix</code> .	<pre>fm(1234567, fmt="engabb", digits=2) ## 1.23 M fm(0.00012, fmt="engabb", digits=0) ## 120 u</pre>
frac	fractions will (try to) convert numbers to fractions. So 0.1 will be displayed as 1/10. See <code>MASS::fractions()</code> .	<pre>fm(c(0.25, 1.75), fmt="frac") ## 1/4 7/4</pre>

A string that is neither one of these codes nor the name of a style does not stop a numeric x from being formatted. `fm()` issues a warning and falls back to the default format, so that a mistyped format in a report function does not take down the whole report. The warning should not be ignored, though: the values are then shown in a different format than intended, e.g. 0.123 instead of 12.3%.

```
fm(1234.5, fmt="xyz")
## Warning: unknown format code 'xyz' in 'fmt'; using the default format
## 1234.5
```

When `fmt = "p"`, numeric input is interpreted as p-values and formatted for reporting in scientific tables and manuscripts. Only values in the interval from 0 to 1 are considered valid; values outside this range are returned as missing.

Very small p-values are reported using a less-than notation based on a configurable reporting threshold (`pThreshold`), which defaults to 0.001. Values below this threshold are not printed numerically, hence exact zeros are never shown. P-values equal to or greater than the threshold are reported numerically using a fixed number of decimal places, controlled by the `digits` argument (3 by default). A p-value equal to one is reported as "1" without trailing decimals.

By default, p-values are printed with a leading zero before the decimal point. APA-style output without a leading zero can be obtained by setting `leadDigits = 0`, which affects only the visual representation. The comparison operator is followed by a space for typographic clarity. Missing values are propagated unchanged.

```
p <- c(0.2, 0.0481, 0.003, 0.00034)
fm(p, fmt="p")
## 0.200 0.048 0.003 < 0.001
fm(p, fmt="p", leadDigits=0)
## .200 .048 .003 < .001
fm(p, fmt="p*")
## 0.200 0.048 * 0.003 ** < 0.001 ***
```

The following shows the boundary cases. Zero and every value below the threshold get the less-than notation, a value exactly at the threshold is printed. A value slightly below one may be rounded to 1.000, whereas only an exact one is printed as “1”:

```
fm(c(0, 0.00099, 0.001, 0.9999, 1, NA), fmt="p")
## < 0.001 < 0.001 0.001 1.000 1 <NA>
```

The significance stars are always derived from the unrounded value. Two p-values that are printed with the same digits can therefore carry different stars:

```
fm(c(0.0496, 0.0504), fmt="p*")
## 0.050 * 0.050 .
```

digits and pThreshold should match. If p-values are to be reported with two decimals only, the threshold has to be raised to 0.01 as well:

```
fm(p, fmt="p", digits=2, pThreshold=0.01)
## 0.20 0.05 < 0.01 < 0.01
```

The formatter is designed for publication-ready output and intentionally avoids exposing machine-level numerical precision, such as machine epsilon values, which are rarely meaningful in scientific reporting.

Finally, `fmt` can be a function. It is called with the non-missing values of `x` and must return exactly one string for each of them; further arguments given in `...` are passed on to it. `naForm`, `zeroForm`, `width` and `align` are still applied by `fm()`:

```
fm(c(1234.567, 0.5, NA),
   fmt=function(z, unit) paste(round(z, 1), unit),
   unit="kg", naForm="-")
## 1234.6 kg 0.5 kg -
```

1.7 Using Styles

In everyday analysis, it is convenient to encapsulate less relevant details from the analysis code. This also applies to format specifications, which can become quite voluminous. Format templates are a proven approach to countering confusion.

Format templates can be created with `style()`. Any of the arguments from `fm()` can be taken to be combined to a list with class “Style”. Such a template can be passed to `fm()` via the argument `fmt`. This allows to store and manage the full format in variables or as options.

Besides the format arguments of `fm()`, `style()` has two more specific arguments, `x` and `label`:

- `x` an existing style, or the name of one, to start from. The format arguments supplied along with it overwrite its settings or are added to them. If no style with the given name can be found in the global environment or in the options, a warning is issued and an empty style is returned.
- `label` a description for the style

So `style()` can either create new styles or derive them from existing ones. The following will define a new format template named “num.sty”. Passed to `fm()` this will result in a number displayed with 2 fixed digits and a comma as big mark:

```
num.sty <- style(digits=2, bigMark=",")
fm(12222.89345, fmt=num.sty)
## 12,222.89
```

This is the same result as if the arguments had been supplied directly, but helps to avoid boilerplate code: `fm(12222.89345, digits=2, bigMark=",")`. Arguments that are supplied in addition to a style override its settings:

```
fm(12222.89345, fmt=num.sty, digits=0)
## 12,223
```

In general, every setting is looked up in this order: first the argument in the call of `fm()`, then the style, then an option, if there is one for this setting (`bigMark` for the big mark, `OutDec` for the decimal mark, `scipen` for sci and `DescToolsX.lang` for lang), and finally the built-in default. NULL always means “not specified”, so it cannot be used to switch off a setting of a style. For this, the neutral value has to be given explicitly:

```
fm(12222.89345, fmt=num.sty, bigMark="")
## 12222.89
```

To modify a style we can provide `style()` with the style, or with its name, and overwrite resp. add format options. The following uses the current version of the numeric format and changes the digits to 1 and the threshold to switch to scientific presentation to numbers $\geq 1e10$ and $< 1e-10$. The original style remains untouched, so the result must be assigned:

```
one.sty <- style(num.sty, digits=1, sci=10)
fm(12222.89345, fmt=one.sty)
## 12,222.9
```

`styles()` returns all style definitions found in the global environment or in the options. Such a style can also be passed to `fm()` by its name. If the same name exists in both places, the style in the global environment is used.

```
fm(12222.89345, fmt="num.sty")
## 12,222.89
```

The styles can be stored as options for convenience. To store a new format we use the default `options()` approach:

```
options(chf.sty = style(digits=2, bigMark="", label="Swiss francs"))
fm(1234567.891, fmt="chf.sty")
## 1'234'567.89
```

```
names(styles())
## "num.sty" "one.sty" "abs.sty" "chf.sty" "num.sty" "per.sty" "pval.sty"
```

Many report functions (e.g. `tOne()` or `desc()`) in **DescToolsX** use default formats that **pharos** stores as options when it is loaded: “abs.sty” for counts, “num.sty” for numeric values, “per.sty” for percentages and “pval.sty” for p-values. Redefining one of them changes the appearance of all reports at once.

2 Formatting Higher Dimensional Objects

2.1 Vectorizing and Objects

The function is vectorized in `x`, so it takes a vector and applies the format to all of its elements. Names are preserved:

```
fm(c(a=1.234, b=5.678), digits=1)
##   a   b
## 1.2 5.7
```

The format arguments, however, are not recycled along `x`. For a vector, a matrix or a table each of them must be a single value, anything longer is rejected. This is a deliberate decision, as a vector of formats that silently wanders through the elements of `x` is more often a mistake than an intention. If the elements really need different formats, they can be formatted one by one:

```
fm(c(1.234, 5.678), digits=c(1, 2))
## Error: 'digits' must be one valid numeric value

mapply(fm, c(1.234, 5.678), digits=c(1, 2))
## "1.2" "5.68"
```

The exceptions are data frames and ftables, where every argument may have one value per column (see 2.4). `fm()` is generic and has methods for the following objects:

<code>vector</code>	numeric, logical, character, factor, Date or POSIXt. Returns a character vector of class <code>noquote</code> .
<code>matrix</code>	returns a character matrix of class <code>noquote</code> with the same dimensions and <code>dimnames</code> .
<code>table</code>	returns a table.
<code>ftable</code>	returns an ftable. The arguments are applied column by column, as for data frames, and may likewise have one value per column.
<code>data.frame</code>	returns a data frame whose columns have been formatted one by one.

2.2 Combinations of multiple numbers

`fmCI()` is a lightweight helper function in **pharos** for formatting confidence intervals. It passes all values through `fm()` and inserts the results into a template, which replaces the recurring combination of `fm()`, `paste()` or `sprintf()` by one call.

fmCI

By default, `fmCI()` interprets the structure of its input. If `x` has length three, it is assumed to contain a point estimate followed by lower and upper confidence limits, and the result is formatted as estimate [lower, upper]. If `x` has length two, it is interpreted as confidence limits only and formatted as [lower, upper]. If `x` is a matrix or a data frame, every row is formatted as one interval.

The `template` argument allows full control over the output layout and overrides the default. As the values are already formatted when they are inserted, the template may only contain `%s` placeholders, one for each value; numeric conversions such as `%.1f` are rejected. A literal percent sign must be doubled.

<code>x</code>	a numeric vector of length 2 or 3. Length 3 is interpreted as point estimate, lower, and upper confidence limits; length 2 as lower and upper confidence limits only. A matrix or data frame with 2 or 3 columns is processed row by row.
<code>template</code>	an optional character string that defines the output format, with one <code>%s</code> for each value. If <code>NULL</code> , a default template is chosen based on the length of <code>x</code> .
<code>...</code>	additional arguments forwarded to <code>fm()</code> , allowing control over rounding, digits, and numeric formatting. They apply to all values alike, so that estimate and limits always share one format.

```
fmCI(c(0.42, 0.30, 0.55))
## "0.42 [0.30, 0.55]"

fmCI(c(0.30, 0.55))
## "[0.30, 0.55]"

fmCI(c(0.42, 0.30, 0.55),
      template="%s (%s to %s)", digits=1)
## "0.4 (0.3 to 0.6)"

fmCI(c(0.123, 0.051, 0.195), fmt="%", digits=1,
      template="%s (95%-CI %s; %s)")
## "12.3% (95%-CI 5.1%; 19.5%)"
```

Missing and infinite limits are not treated specially. They are formatted by `fm()` like any other value, so that `naForm` applies to them:

```
fmCI(c(0.42, NA, NA))
## "0.42 [NA, NA]"
fmCI(c(0.42, NA, NA), naForm="?")
## "0.42 [?, ?]"
fmCI(c(0.42, 0.30, Inf), digits=2)
## "0.42 [0.30, Inf]"
```

With a matrix of estimates and limits, a complete column of a report table is obtained in one call:

```
ci <- t(sapply(split(warpbreaks$breaks, warpbreaks$tension),
                function(z) c(mean(z), t.test(z)$conf.int)))

ci
##      [,1]      [,2]      [,3]
## L 36.38889 28.21024 44.56753
## M 26.38889 21.85312 30.92466
## H 21.66667 17.51305 25.82028

fmCI(ci, digits=1)
##      L      M      H
## "36.4 [28.2, 44.6]" "26.4 [21.9, 30.9]" "21.7 [17.5, 25.8]"
```

2.3 Matrices and Tables

Formatting a matrix results in a matrix again. The entries are padded to one common width, so that the decimal points line up when the matrix is printed.

```
m <- cor(swiss)

# p-values of the pairwise correlation tests
p <- bedrock::pairApply(swiss, function(x, y) cor.test(x, y)$p.value)

# set nonsignificant correlations to 0
m[p > 0.05] <- 0

# use the upper triangular matrix only
m[lower.tri(m, diag = TRUE)] <- NA

# format matrix
fm(m, digits=3, leadDigits=0, naForm=".", zeroForm="-")
##
##      Fertility Agriculture Examination Education Catholic Infant.Mortality
## Fertility      .      .353      -.646      -.664      .464      .417
## Agriculture      .      .      -.687      -.640      .401      -
## Examination      .      .      .      .698      -.573      -
## Education      .      .      .      .      -      -
## Catholic      .      .      .      .      .      -
## Infant.Mortality .      .      .      .      .      .
```

Nonsignificant values have been replaced by “-” and the lower triangular matrix by “.”.

Tables are treated alike and remain tables:

```
tab <- table(cyl = mtcars$cyl, gear = mtcars$gear)
fm(prop.table(tab, 1), fmt="%", digits=1, zeroForm="-")
##      gear
## cyl 3      4      5
## 4  9.1% 72.7% 18.2%
## 6 28.6% 57.1% 14.3%
## 8 85.7%      - 14.3%
```

For an ftable the arguments are applied column by column and may have one value per column. As the cells of an ftable are printed left-justified, align = “\r” is used here to line up the numbers:

```
ft <- ftable(Titanic, row.vars = 1:2)
fm(prop.table(ft, 1), fmt="%", digits=1, align="\r")
##
##      Age      Child      Adult
##      Survived      No      Yes      No      Yes
## Class Sex
## 1st  Male      0.0%      2.8% 65.6% 31.7%
##      Female      0.0%      0.7%  2.8% 96.6%
## 2nd  Male      0.0%      6.1% 86.0%  7.8%
##      Female      0.0%     12.3% 12.3% 75.5%
## 3rd  Male      6.9%      2.5% 75.9% 14.7%
##      Female      8.7%      7.1% 45.4% 38.8%
## Crew Male      0.0%      0.0% 77.7% 22.3%
##      Female      0.0%      0.0% 13.0% 87.0%
```

2.4 data.frames

While in a matrix all cells are normally displayed with the same format instruction, in data frames the individual columns may well have different formats. For data frames every format argument must therefore have either length one, which is then used for all columns, or one value for each column.

```
d <- data.frame(group = c("A", "B", "C"),
               n      = c(1250, 980, 12043),
               mean   = c(12.3456, 9.8, 105.25),
               share   = c(0.0876, 0.0687, 0.8437),
               p       = c(0.2134, 0.0312, 0.00004))

d
##   group    n    mean share    p
## 1    A 1250 12.3456 0.0876 0.21340
## 2    B  980  9.8000 0.0687 0.03120
## 3    C 12043 105.2500 0.8437 0.00004

fm(d, digits=c(0, 0, 2, 3, 3), bigMark="")
##   group    n    mean share    p
## 1    A 1'250 12.35 0.088 0.213
## 2    B  980  9.80 0.069 0.031
## 3    C 12'043 105.25 0.844 0.000
```

A length in between is rejected, which avoids ambiguous partial recycling:

```
fm(d, digits=c(0, 2))
## Error: Argument 'digits' has length 2, but must be 1 or 5
```

Text columns such as group are passed through; arguments that only concern numbers are ignored for them. A format code, however, is not accepted for a text column. Different codes or styles per column are therefore supplied as a list with one entry per column, where NULL stands for no special format:

```
fm(d, fmt=list(NULL, style(digits=0, bigMark=""), style(digits=2), "%", "p"))
##   group    n    mean share    p
## 1    A 1'250 12.35 8.8% 0.213
## 2    B  980  9.80 6.9% 0.031
## 3    C 12'043 105.25 84.4% < 0.001
```

The result is again a data frame. Its columns are character vectors of class noquote.

3 References

(1) Dalgaard P. (2008) Introductory Statistics with R (2nd ed.), New York: Springer.